Algorithmic Methods of Data Mining and Laboratory

Master's Degree in Data Science, Sapienza Università di Roma
Faculty of Information Engineering, Informatics and Statistics

Department of Computer Science

Author: Gianmaria Romano

Reference Bibliography

C. Aggarwal: *Data Mining: The Textbook*
R. Zafarani, M. A. Abbasi, and H. Liu: *Social Media Mining: An Introduction*
J. Leskovec, A. Rajaraman, and J. Ullman: *Mining of Massive Datasets*
C. D. Manning, P. Raghavan, and H. Schütze: *Introduction to Information Retrieval*
J. Janssens: *Data Science at the Command Line*

Academic Year 2026/2027

Disclaimer

This document contains notes for the **Algorithmic Methods of Data Mining and Laboratory** course, given by **Professors Anagnastopolous and Siciliano** as part of the Master's Degree in Data Science at **Sapienza Università di Roma** during the **first** semester of Academic Year **2026/2027**.

Note: These notes may not cover all details of each lecture, so it is highly suggested to consult the Professor's official resources and materials.

Important: These notes are free to share and use, but please remember to credit me as the author and do not try to hide/remove this page.

Contacts

For further information or requests, feel free to contact me privately or reach out through my repository: https://github.com/GianmariaRomano/Data-Science-Notes.

License

These documents are distributed under the GNU General Public License 3.0, a form of copyleft intended for use on a manual, textbook or other documents. Material licensed under the current version of the license can be used for any purpose, as long as the use meets certain conditions:

- All previous authors of the work must be attributed.
- All changes to the work must be logged.
- All derivative works must be licensed under the same license.
- The full text of the license, unmodified invariant sections as defined by the author if any, and any other added warranty disclaimers (such as a general disclaimer alerting readers that the document may not be accurate for example) and copyright notices from previous versions must be maintained.
- Technical measures such as DRM may not be used to control or obstruct distribution or editing of the document.

Contents

1 Introduction to data mining **3**
- 1.1 Finding a definition of data mining . . . 3
- 1.2 Understanding data types . . . 3
  - 1.2.1 Numeric data . . . 4
  - 1.2.2 Categorical data . . . 4
  - 1.2.3 Document data . . . 4
  - 1.2.4 Set data . . . 5
  - 1.2.5 Ordered data . . . 5
  - 1.2.6 Graph data . . . 5
- 1.3 Storing data . . . 5

2 Algorithms **6**
- 2.1 Defining algorithms . . . 6
- 2.2 Complexity analysis . . . 7
  - 2.2.1 The "gap problem" in asymptotic analysis . . . 8
- 2.3 Recursive algorithms . . . 9
- 2.4 **NP**-completeness . . . 10
  - 2.4.1 Solving **NP**-complete problems via heuristics . . . 10
  - 2.4.2 Solving **NP**-complete problems via approximation algorithms . . . 10

3 Data relationships **11**
- 3.1 Distance functions . . . 11
  - 3.1.1 The L^p distance family . . . 11
  - 3.1.2 The edit distance . . . 12
- 3.2 Similarity functions . . . 13
  - 3.2.1 The Jaccard index . . . 13

Chapter 1

Introduction to data mining

1.1 Finding a definition of data mining

While no unique definition exists yet, it is possible to view data mining as the use of efficient techniques for the analysis of very large collections of data and the extraction of useful and possibly unexpected patterns in data in order to make decisions across various areas.

Example: Descriptive and predictive data mining

Descriptive data mining aims to identify patterns and relationships within data using unsupervised learning techniques that can be applied to recommendation systems, such as Netflix or Spotify's engines, to analyse patterns in user behaviour.
On the other hand, predictive data mining focuses on using past observations to predict a target variable through supervised learning techniques.
For instance, CERN's *A Large Ion Collided Experiment* analyses heavy-ion collision data to investigate the properties of quark-gluon plasma.

1.2 Understanding data types

Since data can come in different forms, it is necessary to organize it in order to extract meaningful information or patterns.
In particular, data can be seen as a collection of attributes that describe some objects and, depending on the organization paradigm, data can be classified into three main types:

- **Structured data:** Data is organized according to a well-defined schema that is enforced during storage.
- **Semi-structured data:** Data can be organized according to a structural schema, although this organization paradigm may not be enforced during storage.
- **Unstructured data:** Data does not follow a pre-defined structure, requiring a data cleaning procedure to store it as (semi-)structured data before use.

Example: Different data types

Examples of data organization can be seen on the InfoStud platform, which enforces strict organization through dedicated constraints on certain types of data, and medical records, which follow some structural patterns but might contain errors or anomalous data.
By contrast, social media platforms, such as Twitter or Reddit, provide clear examples of unstructured data that do not follow an organizational structure.

1.2.1 Numeric data

If each record has the same fixed set of numeric attribute, it can be seen as a point in a multi-dimensional space, meaning that the dataset can be treated as a matrix.

	#doors	Horsepower	weight (kg)	price	length (m)	Final speed (km/h)	0-100m (sec)
Car 1	3	120	1520	15,000	3.10	195	13.7
Car 2	4	210	1660	29,000	4.22	248	8.2
Car 3	5	158	2100	32,500	4.92	210	11.4

Figure 1.1: Example of numeric record data.

1.2.2 Categorical data

Categorical data consists of a collection of records with a fixed set of categorical variables.

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	High	No
2	No	Married	Medium	No
3	No	Single	Low	No
4	Yes	Married	High	No
5	No	Divorced	Medium	Yes
6	No	Married	Low	No
7	Yes	Divorced	High	No
8	No	Single	Medium	Yes
9	No	Married	Medium	No
10	No	Single	Medium	Yes

Figure 1.2: Example of categorical record data.

1.2.3 Document data

Using the "bag-of-words" representation, a document can be represented using a text vector that indicates the occurrences of certain words of interest within the document.

	team	coach	play	ball	score	game	win	lost	timeout	season
Document 1	3	0	5	0	2	6	0	2	0	2
Document 2	0	7	0	2	1	0	0	3	0	0
Document 3	0	1	0	0	1	2	2	0	3	0

Figure 1.3: Example of document data.

However, text vectors are often vulnerable to the "curse of dimensionality", which denotes all the performance problems that arise when dealing with high-dimensional data.
For this reason, document data are often pre-processed using dimensionality reduction techniques or principal component analysis in order to focus on just the relevant information carried by the data.

1.2.4 Set data

Set data can be seen as an efficient way of representing collections through binary attributes.

TID	*Items*
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

Figure 1.4: Example of set data.

Note that this type of data is often used to compress document data into sets of words.

1.2.5 Ordered data

Ordered data consists of ordered sequences of values that come in handy for genomics or for time-series forecasting.

1.2.6 Graph data

Graph data allows to summarise web network data using a (directed) graph $G(V, E)$ whose nodes represent pages or users and whose edges denote connections, such as links or interactions, between pages or users.

1.3 Storing data

A computer typically features a processor that performs calculations on data that can be stored on its memory or on (external) hard disks.
Most of the time, however, data is stored using a cloud infrastructure that consists of clustered server racks that can store large amounts of data that might not fit on a normal computer.

Chapter 2

Algorithms

2.1 Defining algorithms

An algorithm can be seen as a set of instructions to solve a given problem.
In particular, an algorithm is evaluated based on its correctness, which determines whether the algorithm works on every possible input or not, and its efficiency, which measures the time taken by the algorithm to find a solution to the given problem.

Example: The maximum value of an array

Consider an algorithm that finds the maximum value in an array of n elements:

Algorithm 1: Finding the Maximum Value in an Array.

```
Data: An array A of size n
Result: max(A)
m ← A[1];
for i ← 1 to n do
    if A[i] > m then
        m ← A[i];
return m;
```

The correctness of this algorithm can be proved by induction on the number of steps.

- For the base case, given by $k = 1$, $A[k]$ is trivially the maximum value.
- Assuming the algorithm is correct on an input of size k, the goal is to prove its correctness on an input of size $(k + 1)$ as well.
  Claim that, after k steps, $m = \max_{i \in \{1,\dots,k\}} (A[i])$: it is possible to see that m will be updated if and only if $A[k + 1] > m$.

Additionally, looking at the pseudocode allows to bound the running time of this algorithm to at most $3n + 2$ moves on an input of size n.

Generally speaking, an algorithm is written using pseudocode in order to provide a language-independent idea of the solution to the given problem.

2.2 Complexity analysis

The running time of an algorithm is a function $T(n)$ that describes the number of steps done by the algorithm to solve a problem on an input of size n.
Since algorithm efficiency often depends on the code or machine that is used, it is often sufficient to find an estimate of the running time of an algorithm through asymptotic analysis.

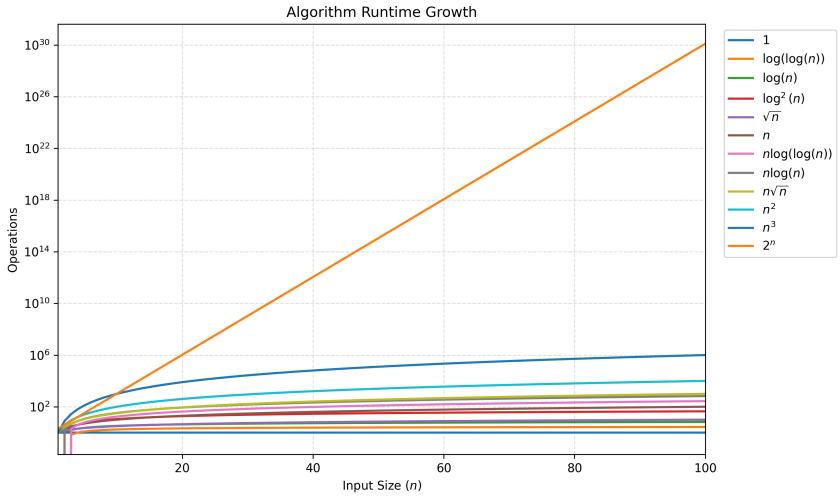


Figure 2.1: Visual comparison of common running time growth rates.

Therefore, given a function $g : \mathbb{N} \to \mathbb{R}^+$, it is possible to define the following asymptotic bounds:

- **Upper bound:** $T(n) = O(g(n))$ if it is possible to find $c > 0$ and $n_0 \geq 0$ such that

$$0 \leq T(n) \leq c \cdot g(n) \ \forall n \geq n_0$$

- **Lower bound:** $T(n) = \Omega(g(n))$ if it is possible to find $c > 0$ and $n_0 \geq 0$ such that

$$0 \leq c \cdot g(n) \leq T(n) \ \forall n \geq n_0$$

- **Tight bound:** $T(n) = \Theta(g(n))$ if it possible to find $c_1, c_2 > 0$ and $n \geq n_0$ such that

$$0 \leq c_1 \cdot g(n) \leq T(n) \leq c_2 \cdot g(n) \ \forall n \geq n_0$$

Example: The prefix sum problem

The following algorithm computes the prefix sum for an array of n elements:

Algorithm 2: Computing the Prefix Sum of an Array.

Data: An array A of size n
Result: An array S of size n such that $S[i] = \sum_{k=1}^{i} A[k]$
$n \leftarrow len(A)$;
for $i \leftarrow 1$ **to** n **do**
    $S[i] \leftarrow 0$;
    for $j = 1$ **to** i **do**
        $S[i] \leftarrow S[i] + A[j]$;
return S;

Since this solution computes $i \leq n$ operations at each iteration, its running time can be bounded as $T(n) = n^2 + 3n + 2 = O(n^2)$.
However, since $T(n) = \Omega(n)$, there is a gap between the lower and upper running time bounds, requiring further investigation to find a better or tighter bound.
For this reason, one can opt for an alternative algorithm that re-uses a previously computed prefix $S[i-1]$ to compute $S[i]$ in constant time.

Algorithm 3: Alternative Algorithm for the Prefix Sum of an Array.

Data: An array A of size n
Result: An array S of size n such that $S[i] = \sum_{k=1}^{i} A[k]$
$n \leftarrow len(A)$;
$S[1] \leftarrow A[1]$;
for $i \leftarrow 2$ **to** n **do**
    $S[i] \leftarrow S[i-1] + A[i]$;
return S;

In fact, this strategy allows to reduce the overall time complexity to the optimal value of $T(n) = \Theta(n)$.

2.2.1 The "gap problem" in asymptotic analysis

The gap problem is a situation in which the lower and upper running time bounds of an algorithm do not match, meaning that no tight bound for the algorithm's complexity can be defined.

Example: Matrix multiplication

Suppose you want to compute the product between two matrices of size $n \times n$: the running time of the standard algorithm has a lower bound of $\Omega(n^2)$, which comes from the need to read and write each element in the product matrix, and an upper bound of $O(n^3)$, which denotes the overall computational cost.
Since the lower and upper bounds do not match, it is not possible to define a tight running time bound for the matrix multiplication algorithm.

2.3 Recursive algorithms

In some cases, it is possible to solve a problem using a recursive algorithm, which makes use of a function that (cyclically) calls itself to solve smaller pieces of a complex problem.

Example: The Fibonacci sequence

It is possible to compute a generic term of the Fibonacci sequence using a recursive algorithm.

Algorithm 4: Recursive Fibonacci.

Data: A non-negative integer n
Result: The nth element of the Fibonacci sequence
if $n \leq 1$ **then**
  return 1;
$f_1 \leftarrow Fibonacci(n-1)$;
$f_2 \leftarrow Fibonacci(n-2)$;
return $f_1 + f_2$;

In particular, the running time of this algorithm can be formulated as

$$T(n) = \begin{cases} 2 & \text{if } n = 0 \text{ or } n = 1 \\ T(n-1) + T(n-2) + 3 & \text{if } n \geq 2 \end{cases}$$

Unrolling this formulation reveals that

$$\begin{aligned} T(n) &= T(n-1) + T(n-2) + 3 \geq T(n-2) + T(n-2) = 2T(n-2) \\ &\geq 2 \cdot 2T(n-4) = 4T(n-4) \geq \cdots \geq 2^i T(n-2i) \geq \dots \\ &\geq 2^{\frac{n}{2}} T\left(n - 2\left(\frac{n}{2}\right)\right) = 2^{\frac{n}{2}} T(0) = 2^{\frac{n}{2}+1} = O(2^{\frac{n}{2}}) \end{aligned}$$

Since the recursive solution runs in exponential time, it can be more convenient to opt for a trivial constant-time algorithm based on Binet's formula or for a linear-time iterative algorithm.

Algorithm 5: Iterative Fibonacci.

Data: A non-negative integer n
Result: The nth element of the Fibonacci sequence
if $n \leq 1$ **then**
  return 1;
previous $\leftarrow 0$;
current $\leftarrow 1$;
for $i \leftarrow 2$ **to** n **do**
  new $\leftarrow$ *previous* + *current*;
  previous $\leftarrow$ *current*;
  current $\leftarrow$ *new*;
return *current*;

2.4 NP-completeness

When studying algorithms and computational complexity, (decision) problems can be classified depending on how hard they are to solve.
For this reason, it is possible to define the complexity classes **P**, which includes all decision problems that can be solved in polynomial time using a deterministic algorithm, and **NP**, which includes all decision problems for which a proposed solution can be verified in polynomial time.
While it is trivially known that $\mathbf{P} \subseteq \mathbf{NP}$, the question on whether $\mathbf{P} = \mathbf{NP}$ remains open.
In this setting, a problem **X** is said to be "**NP**-complete" if $\mathbf{X} \in \mathbf{NP}$ and **X** is **NP**-hard, which means that every problem $\mathbf{Y} \in \mathbf{NP}$ can be reduced to **X** in polynomial time.

Example: The travelling salesman problem

When formulated as a decision problem, the travelling salesman problem aims to determine whether a graph $G = (V, E)$ contains a path that visits all node in the graph while maintaining an overall cost of at most c.
Using some polynomial time reductions, it is possible to prove that this problem is **NP**-complete.

To prove that a problem **X** is **NP**-complete, it is typically sufficient to show that $\mathbf{X} \in \mathbf{NP}$ and that some problem $\mathbf{Y} \in \mathbf{NP}$ can be reduced in polynomial time to **X**.

2.4.1 Solving NP-complete problems via heuristics

Based on "metaheuristics", which represent high-level algorithmic frameworks for problem solving, heuristics provide a problem-specific rule to solve a **NP**-complete problem.
While heuristics are easy to implement and can provide acceptable solutions in reasonable time, they do not guarantee optimality and might struggle to generalize on different instances of the problem.

2.4.2 Solving NP-complete problems via approximation algorithms

In the context of optimization problems, a c-approximation algorithm is an algorithm that is guaranteed to return a solution within a factor c of the optimal solution in polynomial time.

Example: Approximating the travelling salesman problem

While the travelling salesman problem cannot be solved in polynomial time, there are some approximation algorithms that, for a cost function that satisfies the triangle inequality, are guaranteed to find a solution whose value is at most two times the optimal value.

Chapter 3

Data relationships

3.1 Distance functions

Since most data mining problems focus on describing relationships between objects wthin a "universe" $\mathcal{U}$, object discrepancies are measured using a distance function $d : \mathcal{U} \times \mathcal{U} \to \mathbb{R}^+$ such that

1. The function is non-negative, meaning that $d(x, y) \geq 0 \ \forall (x, y) \in \mathcal{U} \times \mathcal{U}$, with $d(x, y) = 0$ if and only if $x = y$.
2. The function is symmetric, meaning that $d(x, y) = d(y, x) \ \forall (x, y) \in \mathcal{U} \times \mathcal{U}$.
3. The function obeys the triangle inequality, meaning that $d(x, y) \leq d(x, z) + d(z, y) \ \forall x, y, z \in \mathcal{U}$.

3.1.1 The L^p distance family

If $\mathcal{U} = \mathbb{R}^m$, the distance between two m-dimensional vectors can be measured using the L^p distance function, which, for an arbitrary $p \in \mathbb{N}$, is defined as

$$d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{j=1}^{m} |\mathbf{x}_j - \mathbf{y}_j|^p \right)^{\frac{1}{p}} = \|\mathbf{x} - \mathbf{y}\|_p$$

As $p \to +\infty$, the L^p distance function becomes the Chebyshev distance, which is defined as

$$d_\infty(\mathbf{x}, \mathbf{y}) = \lim_{p \to +\infty} d_p(\mathbf{x}, \mathbf{y}) = \max_{1 \leq j \leq m} \{|\mathbf{x}_j - \mathbf{y}_j|\} = \|\mathbf{x} - \mathbf{y}\|_\infty$$

Example: Common distances between two vectors

Consider the vectors $\mathbf{x} = \begin{pmatrix} 3 & 7 & 0 & 2 & 1 \end{pmatrix}$ and $\mathbf{y} = \begin{pmatrix} 4 & 5 & 10 & 7 & 3 \end{pmatrix}$: using some common distances from the L^p family results in

$$\begin{aligned} d_1(\mathbf{x}, \mathbf{y}) &= 1 + 2 + 10 + 5 + 2 = 20 \\ d_2(\mathbf{x}, \mathbf{y}) &= 1 + 4 + 100 + 25 + 4 = \sqrt{134} \approx 11.57 \\ d_\infty(\mathbf{x}, \mathbf{y}) &= 10 \end{aligned}$$

Furthermore, choosing $p = 0$ causes the L^p distance to become the Hamming distance, which is defined as

$$d_0(\mathbf{x}, \mathbf{y}) = |\{j : \mathbf{x}_j \neq \mathbf{y}_j\}|$$

3.1.2 The edit distance

A string can be seen as a sequence whose sub-sequence is any sequence that contains a subset of the characters, not necessarily consecutive, of the string: for two strings S and T, a common sub-sequence is any sequence that is a sub-sequence for both S and T.
Using this information, it is possible to define the edit distance between two strings S and T as the minimum number of insertions, deletions and, in some cases, substitutions that are needed to transform S into T, resulting in

$ED(S, T) = |S| + |T| - 2LCS$, where LCS is the length of the longest common sub-sequence.

Example: Computing the edit distance between two strings

Assuming no substitution can be performed, compute the edit distance between the strings $S = CATGA$ and $T = ACTGC$:

$$S = CATGA \overset{del}{\to} ATGA \overset{ins}{\to} ACTGA \overset{ins}{\to} ACTGAC \overset{del}{\to} ACTGC = T$$

Since transforming S into T required four moves, it will be the case that $ED(S, T) = 4$.

While computing the edit distance between two strings typically requires exponential time, it is possible to cut complexity down to polynomial time using a dynamic programming algorithm.

Proof:

Given two strings S and T, of length n and m respectively, define S^i as the sub-sequence containing the first i characters of S and T^j as the sub-sequence containing the first j characters of T: it is possible to prove by induction that the edit distance between S^i and T^j can be recursively computed as

$$ED(S^i, T^j) = \begin{cases} i & \text{if } j = 0 \\ j & \text{if } i = 0 \\ ED(S^{i-1}, T^{j-1}) & \text{if } i, j > 0 \text{ and } S[i] = T[j] \\ 1 + \min\{ED(S^{i-1}, T^j), ED(S^i, T^{j-1})\} & \text{if } i, j > 0 \text{ and } S[i] \neq T[j] \end{cases}$$

Knowing that $T(i, j) = c + T(i - 1, j) + T(i, j - 1)$, let $N = i + j$ and write

$$T(i, j) = T(N) = 2T(N - 1) + c \geq 2T(N - 1) \geq 4T(N - 2) = \cdots = 2^l T(N - l)$$

Since the recursive calls can go on until one of the strings becomes empty, which happens after $l \geq \min\{n, m\}$ steps, the recursive algorithm will require $T(n, m) = \Omega(2^{\min\{n,m\}})$ time. However, by looking at the recursive definition of $ED(S^i, T^j)$, one can implement the recursive call using a $(n + 1) \times (m + 1)$ table of optimal values, resulting in $O(nm)$ complexity.

3.2 Similarity functions

Object similarities can be described using a similarity function $s : \mathcal{U} \times \mathcal{U} \to \mathbb{R}^+$ such that

1. The function is non-negative, meaning that $s(x, y) \geq 0 \ \forall (x, y) \in \mathcal{U} \times \mathcal{U}$, and takes maximum value when $x = y$.
2. The function is symmetric, meaning that $s(x, y) = s(y, x) \ \forall (x, y) \in \mathcal{U} \times \mathcal{U}$.

Additionally, since similarity functions are often normalized, they can be turned into distance functions by setting $d(x, y) = 1 - s(x, y)$.

3.2.1 The Jaccard index

When working with set data, the Jaccard index can be used to measure the similarity between the elements of two sets A and B by computing

$$s(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

In particular, the Jaccard index can be expressed as a distance by setting

$$d(A, B) = 1 - s(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cup B| - |A \cap B|}{|A \cup B|} = \frac{|A| + |B| - 2|A \cap B|}{|A \cup B|}$$